🚀 Collections Framework Masterclass

Chapter 8 — HashSet (Complete Internal Working)

> **Interview Importance:** ⭐⭐⭐⭐⭐

`HashSet` is one of the most commonly asked Collection classes in Java interviews.

If you understand HashSet properly, you will already understand a large part of:

- Hashing
- `hashCode()`
- `equals()`
- Hash collisions
- Buckets
- Load factor
- Resizing
- Treeification
- HashMap internals

And that's why interviewers love it.

---

1. What is HashSet?

`HashSet` is a Set implementation that stores **unique elements** using hashing.

Package:

```
java.util.HashSet
```

Declaration:

```
public class HashSet<E>
    extends AbstractSet<E>
    implements Set<E>,
               Cloneable,
               Serializable
```

Characteristics:

- No duplicate elements
- Allows one `null`
- No guaranteed ordering
- Average O(1) insertion
- Average O(1) deletion
- Average O(1) lookup
- Not synchronized
- Backed internally by a `HashMap`

---

2. The Most Important Fact

HashSet internally uses HashMap

This is the first thing you should remember.

```
HashSet
   ↓
HashMap
   ↓
Hash Table
   ↓
Buckets
```

You don't need to understand HashSet completely separately from HashMap.

Once you understand HashMap internals, HashSet becomes very easy.

---

3. How Does HashSet Store an Element?

Suppose:

```
Set<String> set = new HashSet<>();

set.add("Java");
```

Internally, HashSet effectively does something similar to:

```
map.put("Java", PRESENT);
```

Where:

```
private static final Object PRESENT = new Object();
```

Conceptually:

```
HashSet

Java
  ↓
HashMap

Key        Value
----------------
Java       PRESENT
```

The actual element becomes the **key** of the internal HashMap.

The value is simply a dummy object.

---

4. Why Does HashSet Use HashMap?

Because HashMap already provides:

- Hashing
- Buckets
- Collision handling
- Resizing
- `hashCode()`
- `equals()`
- Treeification

There is no need to implement all of this again.

HashSet simply uses the HashMap infrastructure to enforce uniqueness.

---

5. Simplified HashSet Source Code

Conceptually:

```
public class HashSet<E> {

    private transient HashMap<E,Object> map;

    private static final Object PRESENT =
        new Object();

    public boolean add(E e) {

        return map.put(e, PRESENT) == null;
    }
}
```

This is an extremely important piece of code for interviews.

---

6. What Happens When We Call add()?

Consider:

```
Set<String> set = new HashSet<>();

set.add("Java");
```

Conceptually:

```
set.add("Java")
       ↓
map.put("Java", PRESENT)
       ↓
hash("Java")
       ↓
calculate bucket
       ↓
check bucket
       ↓
insert Java
```

7. Hashing Process

Suppose:

```
set.add("Java");
```

Java obtains:

```
"Java".hashCode()
```

Then HashMap applies its internal hash-spreading function.

Conceptually:

```
Object
  ↓
hashCode()
  ↓
hash spreading
  ↓
bucket index
  ↓
store element
```

The exact HashMap implementation details vary by JDK, but the core idea remains the same.

---

8. What is a Bucket?

A bucket is a location in the internal hash table where entries can be stored.

Imagine:

```
Bucket 0
Bucket 1
Bucket 2
Bucket 3
Bucket 4
Bucket 5
```

```
Bucket 6
Bucket 7
```

An element's hash determines which bucket it belongs to.

Conceptually:

```
"Java"
  ↓
hash
  ↓
bucket 3
```

Then:

```
Bucket 3
   ↓
[Java]
```

---

9. Bucket Index Calculation

A common simplified explanation is:

```
index = hash & (n - 1);
```

Where:

```
n = table length
```

For example:

```
table length = 16
```

Then:

```
index = hash & 15
```

This works efficiently because HashMap capacities are maintained as powers of two.

---

10. Why Power of 2?

Interview Favorite ⭐⭐⭐⭐⭐

HashMap uses table capacities that are powers of two:

```
16
32
64
128
256
...
```

This allows bucket calculation using:

```
hash & (n - 1)
```

instead of the more expensive:

```
hash % n
```

It also works particularly well with the resize algorithm.

---

11. What If Two Objects Go to the Same Bucket?

This is called a:

Hash Collision

Example:

```
Object A → Bucket 5

Object B → Bucket 5
```

Both objects have ended up in the same bucket.

This is completely normal.

A good hash function reduces collisions, but cannot eliminate them.

---

12. Collision Handling

Modern HashMap handles collisions using:

```
Linked List
      ↓
Red-Black Tree
```

depending on the number of entries in a bucket.

Conceptually:

```
Bucket 5

   ↓

[A] → [B] → [C]
```

If the bucket becomes sufficiently crowded, the structure can be transformed into a tree.

```
Bucket 5

      A
     /
    B   C
```

This process is called:

Treeification

We'll study the exact thresholds in the HashMap chapter.

---

13. How Does HashSet Prevent Duplicates?

This is the most important HashSet interview question.

Suppose:

```
Set<String> set = new HashSet<>();

set.add("Java");
set.add("Java");
```

What happens?

First:

```
"Java"
   ↓
hashCode()
   ↓
bucket
   ↓
No matching key
   ↓
Insert
```

Second:

```
"Java"
   ↓
hashCode()
   ↓
same bucket
   ↓
equals()
   ↓
equal
   ↓
Duplicate
   ↓
Do not insert
```

Therefore:

```
set.add("Java");
```

returns:

```
true
```

First time.

Second time:

```
false
```

---

14. hashCode() and equals()

Interview Favorite ⭐⭐⭐⭐⭐

Hash-based collections use:

```
hashCode()
+
equals()
```

Both are important.

The simplified process is:

```
New element
     ↓
hashCode()
     ↓
Find bucket
     ↓
Compare candidates
     ↓
equals()
     ↓
Duplicate?
   ↙       ↘
 Yes       No
```

```
  ↓         ↓
Reject     Insert
```

---

15. Why Do We Need Both?

Suppose we only used `equals()`.

We would potentially have to compare the new object with every object in the Set.

That could become:

```
O(n)
```

for every insertion.

Hashing first narrows the search to a particular bucket.

Then `equals()` performs the final equality check among candidates.

This gives HashSet its excellent average performance.

---

16. Important hashCode() Contract

If two objects are equal:

```
a.equals(b) == true
```

then they **must** have the same hash code:

```
a.hashCode() == b.hashCode()
```

But the reverse is NOT required.

Two different objects can have the same hash code.

That is a collision.

---

17. Same Hash Code Does NOT Mean Equal

Very important.

```
hashCode()
    ↓
Same
    ↓
Collision
    ↓
equals()
    ↓
Could be false
```

Example:

```
Object A
hashCode = 100

Object B
hashCode = 100
```

They can still be different objects.

Therefore:

```
Same hashCode ≠ Same object
```

But:

```
equals() == true
⇒ same hashCode
```

---

18. Custom Object Example

Consider:

```
class Employee {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Now:

```
Set<Employee> employees =
    new HashSet<>();

employees.add(
    new Employee(101, "John")
);

employees.add(
    new Employee(101, "John")
);
```

Many beginners expect:

```
1 Employee
```

But without overriding `equals()` and `hashCode()`:

```
2 Employees
```

can be stored.

Why?

Because `Object`'s default equality is based on object identity.

---

19. Correct Implementation

If Employee identity is based on `id`:

```
class Employee {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {

        if (this == obj)
            return true;

        if (!(obj instanceof Employee))
            return false;

        Employee other =
            (Employee) obj;

        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Now:

```
Set<Employee> employees =
    new HashSet<>();

employees.add(
    new Employee(101, "John")
);

employees.add(
```

```
    new Employee(101, "John")
);
```

Only one logical Employee is stored.

---

20. Why Must hashCode() Be Overridden Too?

Suppose you override only:

```
equals()
```

but not:

```
hashCode()
```

Two logically equal objects may produce different hash codes.

Then they may go into different buckets.

HashSet may never compare them with `equals()`.

Result:

```
Duplicate can be stored
```

Therefore:

> Whenever you override `equals()`, you should normally override `hashCode()` consistently.

---

21. Mutable Keys — Dangerous!

This is an advanced interview question.

Consider:

```
class Employee {
```

```
    int id;

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }

    @Override
    public boolean equals(Object obj) {
        ...
    }
}
```

Then:

```
Employee e = new Employee(101);

Set<Employee> set =
    new HashSet<>();

set.add(e);
```

Suppose we later change:

```
e.id = 200;
```

Now the object's hash code changes.

But the object is still physically located in the bucket determined by the old hash.

Therefore:

```
set.contains(e)
```

may return:

```
false
```

even though the object is physically inside the Set.

---

22. Why?

Before mutation:

```
id = 101
   ↓
hash = H1
   ↓
Bucket 5
```

After mutation:

```
id = 200
   ↓
hash = H2
   ↓
Bucket 11
```

But the object is still sitting in:

```
Bucket 5
```

HashSet searches Bucket 11.

It doesn't find the object.

This is why objects used in hash-based collections should ideally have **stable equality/hash-code fields** while stored.

---

23. Null in HashSet

HashSet allows one `null`.

Example:

```
Set<String> set =
    new HashSet<>();
```

```
set.add(null);
set.add(null);
```

Result:

```
[null]
```

Only one null is stored.

---

24. Why Only One Null?

Because Set does not allow duplicates.

The second null is considered a duplicate.

---

25. Ordering

HashSet does **not guarantee insertion order**.

Example:

```
Set<Integer> set =
    new HashSet<>();

set.add(50);
set.add(10);
set.add(30);
set.add(20);
```

Do not assume output will be:

```
[50, 10, 30, 20]
```

It may appear in a different order.

And importantly:

> Do not rely on the current observed order even if it happens to look predictable.

26. HashSet vs LinkedHashSet

Feature	HashSet	LinkedHashSet
Duplicates	No	No
Insertion Order	No guarantee	Yes
Internal backing	HashMap	LinkedHashMap
Average add	O(1)	O(1)
Average contains	O(1)	O(1)
Memory	Lower	Higher

Use:

```
HashSet
```

when order doesn't matter.

Use:

```
LinkedHashSet
```

when insertion order matters.

27. HashSet vs TreeSet

Feature	HashSet	TreeSet
Ordering	No guarantee	Sorted
Internal Structure	HashMap	Red-Black Tree
Average add	O(1)	O(log n)
contains	O(1) average	O(log n)
Null	One null allowed	Generally not allowed
Range operations	No	Yes

Use HashSet when:

```
Uniqueness + speed
```

Use TreeSet when:

```
Uniqueness + sorted order
```

---

28. Initial Capacity

You can specify initial capacity:

```
Set<Integer> set =
    new HashSet<>(100);
```

This creates a HashSet configured with an initial capacity of 100.

Important:

> Initial capacity is related to the underlying HashMap's table sizing behavior; it does not necessarily mean a 100-element internal array is immediately allocated.

---

29. Load Factor

Interview Favorite ⭐⭐⭐⭐⭐

Default load factor of HashMap, and therefore the backing structure used by HashSet, is:

```
0.75
```

Load factor determines when the hash table should resize.

Conceptually:

```
Threshold =
Capacity × Load Factor
```

Example:

```
Capacity = 16
Load Factor = 0.75

Threshold = 12
```

When the number of entries reaches the resizing threshold, the table is resized.

---

30. Why 0.75?

It's a practical balance between:

```
Memory Usage
        +
Hash Collision Rate
```

Higher load factor:

```
Less memory
More collisions
```

Lower load factor:

```
More memory
Fewer collisions
```

`0.75` is a commonly effective trade-off.

---

31. Resizing

Suppose:

```
Capacity = 16
Load Factor = 0.75
Threshold = 12
```

As the table grows and the threshold is exceeded, HashMap expands the table.

Conceptually:

```
16
 ↓
32
 ↓
64
 ↓
128
```

The entries are redistributed across the larger table.

This operation is called:

Rehashing / Resizing

More precisely, modern HashMap implementations resize and redistribute entries; "rehashing" is commonly used in interview terminology.

---

32. Why Does Resizing Cost More?

Existing entries may need to be redistributed.

Therefore resizing is more expensive than a normal insertion.

A normal insertion:

```
O(1) average
```

Resize:

```
O(n)
```

But resizing happens infrequently.

Therefore average insertion remains:

```
O(1)
```

33. Time Complexity

Operation	Average	Worst Case
add()	O(1)	O(log n)*
remove()	O(1)	O(log n)*
contains()	O(1)	O(log n)*
size()	O(1)	O(1)
iteration	O(n)	O(n)

*For modern Java HashMap buckets that have been treeified. Under certain conditions and pathological/custom scenarios, exact behavior can differ.

---

34. Why O(log n) After Treeification?

Suppose a bucket contains many entries.

Linked list:

```
A → B → C → D → E → F
```

Search:

```
O(n)
```

Tree:

```
        D
       /
     B   F
    /
   A   C
```

Search:

```
O(log n)
```

This protects HashMap/HashSet from pathological collision chains.

---

35. Treeification

Interview Favorite ⭐⭐⭐⭐⭐

When a bucket becomes sufficiently crowded, Java can convert the linked structure into a Red-Black Tree.

Important thresholds in current HashMap implementations include:

```
TREEIFY_THRESHOLD = 8
UNTREEIFY_THRESHOLD = 6
MIN_TREEIFY_CAPACITY = 64
```

The important interview nuance is:

> Reaching 8 entries in a bucket does **not automatically mean treeification**.

If the table is still smaller than `64`, HashMap may resize instead of treeifying.

So the simplified rule is:

```
Bucket becomes highly crowded
        ↓
Check table capacity
        ↓
Capacity < 64?
   ↙            ↘
 Yes            No
 ↓               ↓
Resize        Treeify
```

---

36. Why Doesn't Java Treeify Immediately?

Tree nodes consume more memory than ordinary linked-list nodes.

If the entire table is still small, increasing the table size can reduce collisions more cheaply.

Therefore Java uses resizing first in smaller tables.

---

37. HashSet is Not Thread-Safe

Interview Favorite ⭐⭐⭐⭐⭐

This:

```
Set<Integer> set =
    new HashSet<>();
```

is not thread-safe.

Multiple threads modifying it concurrently can lead to incorrect behavior.

---

38. Making HashSet Synchronized

Option 1:

```
Set<Integer> set =
    Collections.synchronizedSet(
        new HashSet<>()
    );
```

This wraps the HashSet with synchronized access.

---

39. Concurrent Alternative

If you need a highly concurrent Set:

```
Set<Integer> set =
    ConcurrentHashMap.newKeySet();
```

This is generally preferable to synchronizing an entire HashSet when you need scalable concurrent access.

We'll study this in the `ConcurrentHashMap` chapter.

---

40. Fail-Fast Iterator

HashSet iterators are generally **fail-fast**.

Example:

```
Set<Integer> set =
    new HashSet<>();

set.add(10);
set.add(20);
set.add(30);

for (Integer i : set) {

    if (i == 20) {
        set.remove(i);
    }
}
```

This can result in:

```
ConcurrentModificationException
```

because the Set was structurally modified outside the iterator.

---

41. Correct Removal During Iteration

Use Iterator:

```
Iterator<Integer> itr =
    set.iterator();

while (itr.hasNext()) {

    Integer value = itr.next();

    if (value == 20) {
        itr.remove();
```

```
    }
}
```

This is the correct way to remove an element through the iterator.

---

42. Is Fail-Fast Guaranteed?

Important interview nuance:

No.

Fail-fast behavior is a best-effort mechanism.

You should not build application logic that depends on `ConcurrentModificationException`.

---

43. HashSet and Streams

HashSet works naturally with Stream API.

Example:

```
Set<Integer> numbers =
    new HashSet<>();

numbers.add(10);
numbers.add(20);
numbers.add(30);

numbers.stream()
       .filter(n -> n > 10)
       .forEach(System.out::println);
```

---

44. Removing Duplicates from a List

One of the most common real-world uses of HashSet.

```
List<Integer> numbers =
    Arrays.asList(
```

```
        10, 20, 10, 30, 20
    );

Set<Integer> unique =
    new HashSet<>(numbers);
```

Result:

```
[10, 20, 30]
```

Remember:

The resulting HashSet does not guarantee the original List's order.

If order must be preserved:

```
Set<Integer> unique =
    new LinkedHashSet<>(numbers);
```

---

45. Real-World Use Cases

Unique User IDs

```
Set<Long> userIds;
```

Useful for preventing duplicate IDs.

Unique Permissions

```
Set<String> permissions;
```

Example:

```
READ
WRITE
DELETE
```

Unique Tags

```
Set<String> tags;
```

Duplicate Detection

```
Set<String> seen =
    new HashSet<>();

if (!seen.add(value)) {
    // duplicate
}
```

This is a very useful pattern.

Why?

Because `add()` returns:

```
true  → new element
false → duplicate
```

---

46. Duplicate Detection Pattern

Example:

```
Set<Integer> seen =
    new HashSet<>();

for (Integer number : numbers) {

    if (!seen.add(number)) {
        System.out.println(
            "Duplicate: " + number
        );
```

```
    }
}
```

This gives average O(n) duplicate detection across the entire collection.

Much better than repeatedly comparing every pair, which can become $O(n^2)$.

---

47. HashSet with Custom Objects

Example:

```
class Employee {

    private final int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public boolean equals(Object obj) {

        if (this == obj)
            return true;

        if (!(obj instanceof Employee))
            return false;

        Employee other =
            (Employee) obj;

        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Now:

```
Set<Employee> employees =
    new HashSet<>();

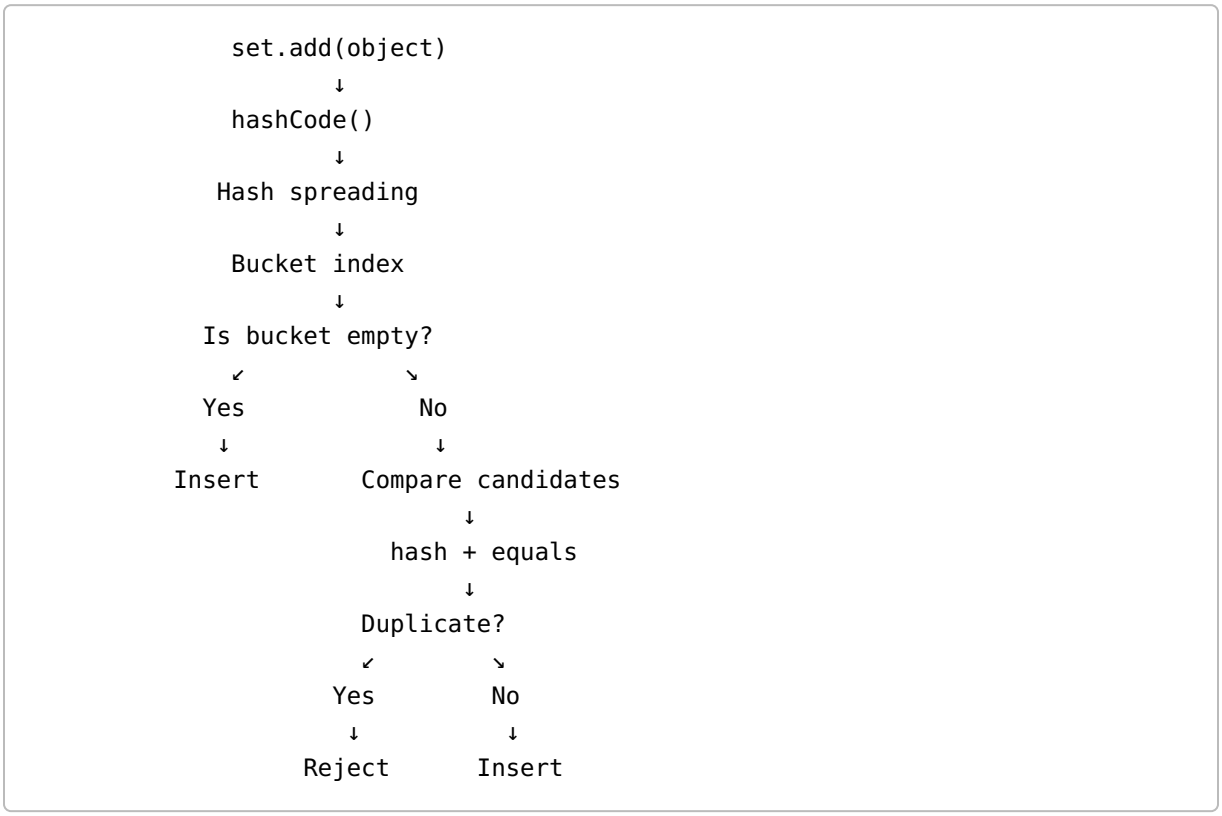
employees.add(new Employee(101));
employees.add(new Employee(101));
```

Only one logical Employee exists in the Set.

---

48. The Most Important HashSet Flow

Memorize this.



This flow is the foundation for understanding HashMap.

---

49. Important Interview Trap

Question:

> If two objects have the same hashCode(), will HashSet consider them duplicates?

Answer:

No.

Same hash code only means:

```
Possible collision
```

HashSet still needs equality comparison.

Correct answer:

```
Same hashCode
        ↓
Same bucket
        ↓
equals()
        ↓
true  → duplicate
false → different objects
```

---

50. Another Interview Trap

Question:

> If two objects have different hashCodes(), can equals() return true?

According to the `equals()` / `hashCode()` contract:

No.

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

must also be true.

---

51. HashSet vs ArrayList for contains()

Suppose:

```
List<Integer> list =
    new ArrayList<>();
```

and:

```
Set<Integer> set =
    new HashSet<>();
```

For:

```
contains(500000)
```

ArrayList:

```
O(n)
```

HashSet:

```
O(1) average
```

Therefore, if you frequently need membership checks and ordering/indexing isn't required, HashSet is often the better choice.

---

52. When Should You Use HashSet?

Choose HashSet when:

```
Need uniqueness
        +
Need fast lookup
```

```
        +
Order doesn't matter
```

Examples:

```
Unique IDs
Unique usernames
Unique permissions
Visited nodes
Already processed records
Duplicate detection
```

---

53. When Should You NOT Use HashSet?

Don't use HashSet when:

You need duplicates

Use:

```
List
```

You need insertion order

Use:

```
LinkedHashSet
```

You need sorted order

Use:

```
TreeSet
```

You need index-based access

Use:

```
ArrayList
```

---

54. HashSet Interview Questions

Q1. How does HashSet work internally?

HashSet is backed by a HashMap. Elements are stored as keys, while a dummy object is used as the value.

Q2. Why doesn't HashSet allow duplicates?

HashSet uses the underlying HashMap's hashing and equality mechanism. If an equivalent key already exists, the new element is not inserted.

Q3. What methods are used to detect duplicates?

```
hashCode()
equals()
```

Q4. Can two objects have the same hashCode?

Yes.

That's called a hash collision.

Q5. Can two equal objects have different hashCodes?

No.

That violates the `equals()` / `hashCode()` contract.

Q6. Can HashSet contain null?

Yes.

It can contain one `null`.

Q7. Is HashSet ordered?

No ordering guarantee.

Q8. Is HashSet thread-safe?

No.

Q9. Average complexity of HashSet.contains()?

O(1).

Q10. What happens during a hash collision?

Multiple entries can occupy the same bucket. Modern HashMap-based implementations use linked structures and may treeify heavily populated buckets.

Q11. What is load factor?

A threshold ratio that determines when the underlying hash table should resize.

Default:

```
0.75
```

Q12. What is the default initial capacity?

The underlying HashMap uses a default initial capacity of **16** when it first allocates its table, although a default-constructed HashMap/HashSet does not immediately allocate a 16-element table.

Q13. What is treeification?

Converting a heavily populated collision bucket from a linked structure into a Red-Black Tree.

Q14. Why doesn't HashSet guarantee order?

Because its iteration order depends on the underlying hash table structure, hash distribution, resizing, and implementation details.

Q15. How do you preserve insertion order while maintaining uniqueness?

Use:

```
LinkedHashSet
```

---

55. ⭐⭐⭐⭐⭐ Interview Scenario

Question:

You have 10 million customer IDs.

You need to:

1. Remove duplicates
2. Frequently check whether an ID exists
3. Don't care about ordering

Which collection?

Answer:

```
HashSet<Long>
```

Why?

```
Unique values
+
Fast average lookup
+
No ordering requirement
```

---

56. ⭐⭐⭐⭐⭐ Interview Scenario

Question:

You need to remove duplicates from a List but preserve the original insertion order.

Answer:

```
List<Integer> result =
    new ArrayList<>(
        new LinkedHashSet<>(numbers)
    );
```

Why?

```
LinkedHashSet
    ↓
Uniqueness
+
Insertion Order
```

---

57. ⭐⭐⭐⭐⭐ Interview Scenario

Question:

You added an Employee to HashSet.

Later you changed the Employee's ID.

Now `contains(employee)` returns false.

Why?

Because the fields participating in `equals()` / `hashCode()` changed.

The object's new hash code points to a different bucket from the one where it was originally stored.

---

58. Golden Rules

Remember these five rules:

Rule 1

```
HashSet → HashMap internally
```

Rule 2

```
Element → HashMap Key
```

Rule 3

```
Duplicate detection → hashCode() + equals()
```

Rule 4

```
Average operations → O(1)
```

Rule 5

```
No ordering guarantee
```

🧠 Final Mental Model

Think of HashSet as:

```
                HashSet
                   ↓
                HashMap
                   ↓
             Hash Table
                   ↓
                Buckets
                   ↓
              hashCode()
```

```
                ↓
          Bucket Index
                ↓
           Collision?
            ↙       ↘
          No         Yes
          ↓           ↓
       Insert      Compare
                      ↓
                   equals()
                  ↙       ↘
               Equal     Different
                 ↓          ↓
              Reject      Insert
```

Once this mental model is clear, **HashMap internals become much easier**.

🔥 Chapter Summary

Concept	HashSet
Interface	Set
Internal backing	HashMap
Duplicates	Not allowed
Null	One allowed
Ordering	No guarantee
add()	O(1) average
contains()	O(1) average
remove()	O(1) average
Thread-safe	No
Duplicate detection	hashCode() + equals()
Default load factor	0.75
Collision handling	Linked structure / Tree
Treeification	Yes, under appropriate conditions
Best use	Unique + fast lookup

🎯 Interview Readiness Checklist

Before moving ahead, you should be able to answer these **without looking at notes**:

- [ ] How does HashSet work internally?
- [ ] Why does HashSet use HashMap?
- [ ] How are duplicates detected?
- [ ] Why are both `hashCode()` and `equals()` required?
- [ ] What is a hash collision?
- [ ] Can different objects have the same hash code?
- [ ] Can equal objects have different hash codes?
- [ ] What is a bucket?
- [ ] What is load factor?
- [ ] What happens during resizing?
- [ ] What is treeification?
- [ ] Why is HashSet O(1) on average?
- [ ] Why isn't HashSet thread-safe?
- [ ] Why can a mutable object become "lost" inside HashSet?
- [ ] HashSet vs LinkedHashSet?
- [ ] HashSet vs TreeSet?
- [ ] How do you remove duplicates while preserving order?

If you can answer all of these confidently, you're ready for the next major topic.

🚀 Next Chapter

Chapter 9 — LinkedHashSet (Complete Deep Dive)

We'll cover:

- How LinkedHashSet preserves insertion order
- HashSet vs LinkedHashSet internally
- LinkedHashMap relationship
- Doubly linked list inside the hash table
- `before` and `after` pointers
- Insertion-order iteration
- Performance and memory trade-offs
- Why LinkedHashSet is slightly more expensive
- Duplicate detection
- Real-world use cases
- 20+ interview questions